University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

# MoveLink

Visualisierung von Echtzeit-Bewegungsdaten

## Pflichtenheft

Software-Architekturen Labor

Sommersemester 2026

**Betreuer:** Prof. Dr. rer. nat. Sinz

**Team:** Erlind Sejdiu
Luca Schöneberg
Devin Uyan

**Abgabe:** 08. April 2026

Version 1.0 – Stand: 8. April 2026

# 1 Grundlegende Problemstellung

Im Krafttraining ist eine korrekte Bewegungsausführung essenziell für den Trainingserfolg und die Verletzungsprävention. Dennoch fehlt es den Trainierenden in der Praxis meist an unmittelbarem, objektivem Feedback zur Qualität ihrer Ausführung.

Zwar können Bewegungsabläufe durch moderne Hardware-Sensoren erfasst werden, jedoch liegen diese Informationen oft nur als unstrukturierte Rohdaten vor, die für den Endanwender kaum interpretierbar sind. Daraus ergibt sich die Herausforderung, die anfallenden Datenströme aus verschiedenen Quellen zentral zusammenzuführen und visuell nutzergerecht aufzubereiten.

Um diese Lücke zu schließen, wird im Rahmen dieses Laborprojekts die mobile Applikation MoveLink konzipiert und prototypisch entwickelt. Ziel der Anwendung ist es, als zentrale Schnittstelle zu fungieren, die Trainingsdaten von unterschiedlichen Hardware-Sensoren standardisiert erfasst, speichert und in einer einheitlichen Benutzeroberfläche verständlich darstellt.

# 2 Use Cases

Bevor eine Architekturerläuterung folgt, sollen zunächst die folgenden Use Cases definiert werden. Diese dienen dem Verständnis der Anforderungen an das System. Jeder Use Case wird durch eine kurze Beschreibung und nachfolgend durch eine Auflistung der Schritte, die zur Erfüllung des Use Cases notwendig sind, definiert.

**UC-1: Echtzeit-Training überwachen**
*Akteur:* Trainierender
*Vorbedingung:* Trainingsgerät verbunden
*Beschreibung:* Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.

1. Trainierender startet eine Trainingseinheit in der App.
2. Sensor erfasst Bewegungsdaten und sendet sie an die Applikation.
3. Frontend visualisiert die Bewegung in Echtzeit.

**UC-2: Trainingsgerät verbinden**
*Akteur:* Trainierender
*Vorbedingung:* Trainingsgerät eingeschaltet
*Beschreibung:* Ich als Trainierender möchte mein Trainingsgerät mit der App verbinden können, um Trainingsdaten erfassen zu können.

1. App zeigt verfügbare Sensoren an.
2. Trainierender wählt Sensor aus; Verbindung wird hergestellt.

**UC-3: Trainingsdaten einsehen**
*Akteur:* Trainierender
*Vorbedingung:* Vergangene Trainingseinheit vorhanden
*Beschreibung:* Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.

1. Trainierender navigiert zur Übersicht vergangener Einheiten.
2. App zeigt historische Bewegungsdaten an.

## 3 Systemanforderungen (App-Fokus)

Aus den oben definierten Use Cases ergeben sich die folgenden Anforderungen an die Applikation, unterteilt in funktionale und nicht-funktionale Kriterien.

### Funktionale Anforderungen

**Muss-Kriterien:**

- Die App muss eine Verbindung zu den Trainingssensoren aufbauen und aufrechterhalten können.
- Die App muss Echtzeit-Datenströme empfangen und verarbeiten können.
- Die App muss eine grafische Benutzeroberfläche zur Live-Visualisierung der Bewegungsdaten bereitstellen.
- Die App muss eine Historien-Ansicht zum Abrufen und Anzeigen vergangener Trainingseinheiten anbieten.

**Kann-Kriterien:**

- Die App kann neben der visuellen Darstellung auch Feedback (z. B. Warntöne bei falscher Haltung) ausgeben.
- Die App kann detaillierte statistische Auswertungen (z. B. Fortschrittskurven über mehrere Wochen) in der Historie anzeigen.

### Nicht-funktionale Anforderungen

**Muss-Kriterien:**

- **Latenz/Performance:** Die Visualisierung der Sensordaten in der App muss in Echtzeit erfolgen (z. B. Latenz $< 100\,\text{ms}$), um ein nutzbares Feedback zu garantieren.
- **Zuverlässigkeit:** Bei einem Verbindungsabbruch zum Sensor oder Server muss die App den Nutzer umgehend benachrichtigen und automatische Reconnect-Versuche starten.
- **Usability:** Der Koppelungsprozess (Pairing) in der App muss intuitiv gestaltet sein und darf maximal zwei Klicks erfordern.
- **Plattformunabhängigkeit:** Die App kann als Cross-Platform-Anwendung (z. B. Flutter oder React Native) entwickelt werden, um sowohl iOS- als auch Android-Nutzer zu bedienen.

**Kann-Kriterien:**

- **Ressourceneffizienz:** Die App kann so optimiert werden, dass die Echtzeit-Visualisierung auch auf älteren Smartphones den Akku nicht übermäßig beansprucht.

# 4 Architekturvorschlag

## 4.1 Gewählte Architektur: Containerisierte App-Architektur

Die Architektur konzentriert sich auf die Applikationsschicht und folgt einem dreischichtigen Container-Modell, das modular aufgebaut ist und sich optimal für ein späteres Deployment eignet:

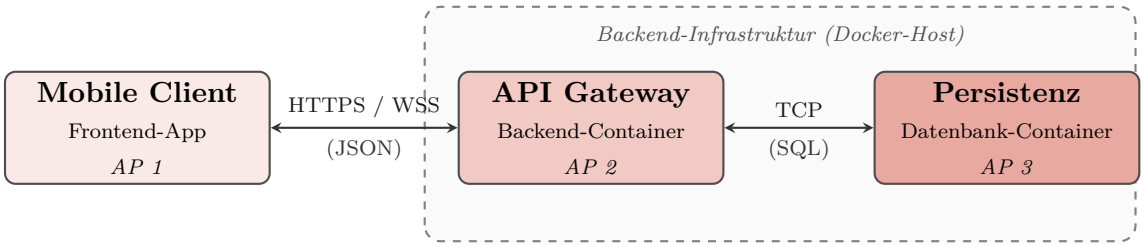


Abbildung 1: Übersicht der Systemarchitektur, Protokolle und Arbeitspakete (AP)

- **Frontend Container:** Stellt die Benutzeroberfläche für den Trainierenden bereit. Diese Schicht ist für die Echtzeit-Visualisierung der Bewegungsdaten und die Koppelung mit externen Sensoren zuständig.

- **Backend Container:** Ein isolierter Server, der als zentrale API und Gateway fungiert. Er verarbeitet Echtzeit-Datenströme und bietet Schnittstellen für statische Anfragen (z. B. Authentifizierung, Abruf von Historien).

- **Datenbank Container:** Kapselt die Datenhaltung. Hier werden Nutzerprofile sowie die in *UC-3* geforderten historischen Trainingsdaten sicher und persistent gespeichert.

## 4.2 Diskussion von Alternativen

Für die Umsetzung des Projekts steht ein Zeitbudget von insgesamt rund 180 Stunden zur Verfügung (ca. 60 Stunden pro Person). Da bereits die Einarbeitung in die verwendeten Technologien einen Teil dieser Zeit beansprucht, ist eine effiziente Planung wichtig. Um in diesem begrenzten Rahmen einen funktionsfähigen Prototypen zu entwickeln, muss die Architektur so aufgebaut sein, dass Aufgaben möglichst unabhängig und parallel bearbeitet werden können.

Aus diesem Grund scheiden sehr komplexe Architekturmuster wie die Clean Architecture oder die Onion Architecture für diesen konkreten Anwendungsfall aus. Der strukturelle Overhead und die anfängliche Setup-Zeit dieser Muster stehen nicht im Verhältnis zum verfügbaren Zeitbudget. Stattdessen fällt die Entscheidung auf einen pragmatischen Architekturansatz. Dieser beschränkt die Komplexität auf das Nötigste, erlaubt schnelle Entwicklungszyklen und stellt sicher, dass der Fokus voll und ganz auf der Umsetzung der wesentlichen Kernfunktionen liegt.

Zur Evaluierung der optimalen Systemarchitektur wurden drei alternative Architekturmuster für den geplanten Anwendungsfall untersucht:

- Client-Server-Architektur (Microservices)
- Client-Server-Architektur (Monolith)
- Publisher-Subscriber-Architektur (IoT-Fokus)

| Architekturansatz | Vorteile | Nachteile |
|---|---|---|
| **Client-Server (Microservices)**<br>Entkoppelte Bereitstellung von Frontend und mehreren Backend-Diensten auf getrennten Systemen. | • Strikte *Separation of Concerns*<br>• Unabhängige Skalierbarkeit der einzelnen Dienste<br>• Hohe technologische Flexibilität | • Hoher Infrastruktur- und Deployment-Aufwand<br>• Mögliche Netzwerklatenzen zwischen den Services<br>• Erhöhte Komplexität für einen Proof of Concept (PoC) |
| **Client-Server (Monolith)**<br>Zentrale Verarbeitungseinheit, die die gesamte Backend-Logik in einer Anwendung bündelt. | • Sehr geringe Latenz (native Performance durch In-Memory-Aufrufe)<br>• Einfaches Deployment und Testing<br>• Geringer Overhead; ideal für initiale PoC-Entwicklung | • Eingeschränkte, nur vertikale Skalierbarkeit<br>• Starke Kopplung erschwert zukünftige Wartbarkeit bei Systemwachstum<br>• *Single Point of Failure* |
| **Publisher-Subscriber**<br>Ereignisgesteuerte Architektur (z. B. via MQTT) zur asynchronen Verteilung von Nachrichten. | • Hervorragende Skalierbarkeit der Endpunkte<br>• Ideal für kontinuierliche IoT-Sensordaten-Streams<br>• Asynchrone und vollständig entkoppelte Kommunikation | • Zusätzliche Infrastruktur (Message Broker) zwingend notwendig<br>• Überproportionaler Overhead für erste Prototypen<br>• Komplexe Fehlerverfolgung (Debugging) über Systemgrenzen hinweg |

Tabelle 1: Vergleich der evaluierten Architekturmuster für das Fitness-Tracking-System

**Begründung:**

Unter Abwägung der strikten Ressourcenrestriktionen (180 Stunden) und der funktionalen Systemanforderungen erweist sich eine **monolithische Client-Server-Architektur** als die zielführendste Lösung für diesen Proof of Concept (PoC).

Aufgrund der limitierten Entwicklungszeit ist eine Minimierung des Infrastruktur- und Deployment-Overheads erforderlich. Containerisierte Ansätze oder Microservices erzwingen zwar eine logische Trennung, verbrauchen jedoch durch das komplexe Setup wertvolle Projektstunden, die für die Echtzeit-Visualisierung benötigt werden. Auch eine Publisher-Subscriber-Architektur scheidet aus, da die zwingende Integration eines Message Brokers den Rahmen des PoC sprengen würde.

Ein monolithisches Backend bündelt die API-Logik, Datenverarbeitung und Datenbankzugriffe in einer Codebase. Dies eliminiert Netzwerk-Overhead zwischen Services und reduziert die Latenz durch direkte In-Memory-Aufrufe, was den strengen Echtzeit-Anforderungen (Latenz $< 100\,\text{ms}$) zugutekommt. Potenzielle Code-Konflikte bei der parallelen Entwicklung im dreiköpfigen Team werden durch eine saubere interne Strukturierung (Modularer Monolith) vermieden. So kann der Fokus vollständig auf der funktionalen Umsetzung liegen.

## 5 Technologieauswahl

Die Auswahl der Technologien orientiert sich maßgeblich an den definierten Ressourcenrestriktionen (Zeiteffizienz) sowie der Notwendigkeit, die Arbeitspakete klar voneinander zu entkoppeln. Im Fokus stehen Frameworks, die eine schnelle Prototypenentwicklung (MVP) unterstützen, ohne Kompromisse bei der Echtzeitfähigkeit einzugehen.

| Architekturschicht | Technologie | Entscheidungsbegründung |
|---|---|---|
| **Mobile Client** (Frontend) | Cross-Platform Framework (z. B. React Native oder Flutter) | • **Zeiteffizienz:** Single-Codebase für iOS und Android.<br>• **BLE-Support:** Etablierte Bibliotheken für Bluetooth GATT.<br>• **UI-Performance:** Hohe Rendering-Performance für Live-Charts (z. B. via Recharts/Skia). |
| **API & Gateway** (Backend) | Node.js (TypeScript) oder Python (FastAPI) | • **Asynchronität:** Hervorragendes Handling von Non-Blocking I/O für parallele Live-Streams.<br>• **Ökosystem:** Native Integration von WebSocket- und REST-Protokollen. |
| **Persistenz** (Datenbank) | Relationales RDBMS (z. B. PostgreSQL) | • **Datenintegrität:** ACID-konforme Speicherung von Nutzer- und Session-Daten.<br>• **Flexibilität:** JSONB-Support ermöglicht das effiziente Speichern schemaloser Sensordaten (Zeitreihen). |
| **Echtzeit-kommunikation** | WebSockets (WSS) | • **Latenz:** Bidirektionaler Vollduplex-Kanal stellt Feedback-Latenzen von $< 100$ ms sicher.<br>• **Overhead:** Geringerer Header-Overhead im Vergleich zu HTTP-Polling. |
| **Infrastruktur & Containerisierung** | Docker & Docker Compose | • **Isolation:** Fehlerfreie, reproduzierbare lokale Entwicklung ("Works on my machinePrävention).<br>• **Orchestrierung:** Simples, gemeinsames Starten aller Backend-Services mit einem einzigen Befehl. |

Tabelle 2: Übersicht und Begründung des gewählten Technologie-Stacks